

GROWTHLIFT DIGITAL

Week 2 Guide

Advanced CSS & JavaScript Essentials

Self-paced · Remote · 5 days of tasks, tutorials & projects

CSS Grid

Animations

JavaScript

DOM API

Fetch API

YOUR 5-DAY JOURNEY

DAY 1

CSS Grid Layout

DAY 2

Animations & Effects

DAY 3

JS Fundamentals

DAY 4

DOM Manipulation

DAY 5

Fetch API & Review

WEEK 2 DELIVERABLES

- ◆ Responsive CSS Grid layout added to your existing project
- ◆ Hover animations and @keyframes fadeInUp on hero heading
- ◆ Working Tab Switcher built with pure JavaScript and DOM events
- ◆ Random Quote Generator app using the Fetch API
- ◆ All work committed to GitHub and GitHub Pages site updated

PREREQUISITE

Week 2 builds directly on your Week 1 project. Make sure you have: a working index.html, a style.css linked to it, a script.js file ready, and your project pushed to GitHub.

FOUNDER

Abdul Rehman

0328 0055453

CTO

Asad Riaz

growthliftpk@gmail.com

WEBSITE

GrowthLift

growthlift-digital.vercel.app

DAY 1

CSS Grid Layout

DAY 2

Animations & Effects

DAY 3

JS Fundamentals

DAY 4

DOM Manipulation

DAY 5

Fetch API & Review

Last week you learned Flexbox — great for 1D layouts (rows OR columns).
CSS Grid handles 2D layouts (rows AND columns together). Today you will master it.

YOUR TO-DO LIST — DAY 1

- ☐ Open your Week 1 project in VS Code — we will build on top of it
- ☐ Watch the CSS Grid Crash Course video (link below) — pause and code along
- ☐ In your style.css, create a new section called `.grid-section {}`
- ☐ Apply: `display: grid;` inside that selector
- ☐ Set columns using: `grid-template-columns: repeat(3, 1fr);` — this creates 3 equal columns
- ☐ Set spacing using: `gap: 24px;` — adds space between all grid items
- ☐ In your index.html, add a new `<section class="grid-section">` with 6 `<div>` cards inside
- ☐ Give each card a background color, padding: 20px, and border-radius: 8px
- ☐ Now try: `grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));` — watch it become responsive
- ☐ Try spanning a card across 2 columns: `grid-column: span 2;` on the first card
- ☐ Try spanning a card across 2 rows: `grid-row: span 2;` on another card
- ☐ Open Chrome DevTools → click the "grid" badge next to your element to see the grid lines overlay
- ☐ Compare your Grid layout vs your Flexbox layout — understand the difference
- ☐ Commit your work: `git add . && git commit -m "Day 8: CSS Grid layout"`
- ☐ Post a screenshot of your Grid section in the WhatsApp community: "Day 8 Done 🟩"

LEARNING RESOURCES — DAY 1



CSS Grid Crash Course

youtube.com/watch?v=0xMQfnTU6oo

Traversy Media — full CSS Grid tutorial, 40 minutes



Grid vs Flexbox

youtube.com/watch?v=hs3piaN4b5I

Web Dev Simplified — when to use which layout

CS

CSS Grid Gardencssgridgarden.com

Interactive game — learn Grid by growing a garden (28 levels)

MD

MDN CSS Griddeveloper.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout

Complete Grid reference — bookmark this

MENTOR TIP — GRID vs FLEXBOX

Rule of thumb: Use **Flexbox** for navigation bars, button groups, and single-row/column layouts.
Use **Grid** for full page layouts, card grids, and anything that needs rows AND columns.
In real GrowthLift projects, we use both together — Grid for the page shell, Flexbox inside components.

WEEK 2 — WHAT YOU LEARNED TODAY

- Learned CSS Grid — rows, columns, gap, auto-fit, minmax()
- Built a responsive 3-column card grid that adjusts automatically to screen size
- Used grid-column: span and grid-row: span to control card sizes
- Practiced with CSS Grid Garden game — solidified the concepts interactively

DAY 1

CSS Grid Layout

DAY 2

Animations & Effects

DAY 3

JS Fundamentals

DAY 4

DOM Manipulation

DAY 5

Fetch API & Review

Static websites feel outdated. Today you will add motion — hover effects, smooth transitions, and keyframe animations that make your pages feel alive and professional.

YOUR TO-DO LIST — DAY 2

☐ Open your project — we will add animations on top of yesterday's Grid layout☐ Watch the CSS Animations & Transitions tutorial (link below)☐ Step 1 — Add a hover transition to your nav links:

→ Add to your nav a {} selector: **transition: color 0.3s ease, background 0.3s ease;**

→ Add a nav a:hover {} with a different color and background

☐ Step 2 — Add a hover effect to your Grid cards:

→ Add to .grid-section div {}: **transition: transform 0.3s ease, box-shadow 0.3s ease;**

→ Add .grid-section div:hover {}: **transform: translateY(-6px); box-shadow: 0 12px 32px rgba(0,200,100,0.15);**

☐ Step 3 — Create a CSS keyframe animation for your hero section heading:

→ Add to your CSS: **@keyframes fadeInUp { from { opacity: 0; transform: translateY(30px); } to { opacity: 1; transform: translateY(0); } }**

→ Apply to your h1: **animation: fadeInUp 0.8s ease forwards;**

☐ Step 4 — Add a pulsing dot animation (like a live indicator):

→ Create a small `<span class="dot"></span>` in your HTML

→ Style it: width: 10px, height: 10px, border-radius: 50%, background: #00C864

→ Add keyframe: **@keyframes pulse { 0%,100% { transform: scale(1); opacity:1; } 50% { transform: scale(0.6); opacity:0.4; } }**

→ Apply: **animation: pulse 2s ease-in-out infinite;**

☐ Test all animations in Chrome — open DevTools → Animations tab to slow them down☐ Commit: **git add . && git commit -m "Day 9: CSS animations and transitions"**☐ Post a screen-recording or GIF in the community: "Day 9 Done ■"

LEARNING RESOURCES — DAY 2

YT

CSS Transitions

youtube.com/watch?v=8kK-cA99SA0

Traversy Media — CSS Transitions tutorial

YT

CSS Keyframe Animations

youtube.com/watch?v=f1WMjDx4snI

Kevin Powell — CSS Animations from scratch

AN

Animate.css

animate.style

Ready-made CSS animation classes — use in your projects

DV

DevTools Animations

developer.chrome.com/docs/devtools/css/animations

Debug and replay animations in Chrome DevTools

MENTOR TIP — ANIMATION RULES

Keep animations under **0.4 seconds** for UI interactions (hover, click). Longer feels sluggish.

Always use **ease** or **ease-in-out** — never linear (it feels robotic).

Only animate **transform** and **opacity** for performance. Animating width/height causes layout reflows.

WEEK 2 — WHAT YOU LEARNED TODAY

- Added smooth hover transitions to navigation links and cards
- Created a fadeInUp keyframe animation for the hero heading
- Built a pulsing dot animation using @keyframes
- Learned to debug animations in Chrome DevTools Animations panel

DAY 1

CSS Grid Layout

DAY 2

Animations & Effects

DAY 3

JS Fundamentals

DAY 4

DOM Manipulation

DAY 5

Fetch API & Review

HTML is structure. CSS is style. JavaScript is the brain — it makes things happen.
Today you learn the core JS concepts every developer uses every single day.

YOUR TO-DO LIST — DAY 3

- ☐ Create a new file called **script.js** in your project folder
- ☐ Link it to your HTML: add **<script src="script.js" defer></script>** inside **<head>**
- ☐ Open script.js and open your browser console (F12 → Console tab)
- ☐ VARIABLES — type and run in console to understand the difference:
 - **let name = "Your Name";** — can be reassigned
 - **const site = "GrowthLift";** — cannot be reassigned (use for values that don't change)
 - **var oldWay = "avoid this";** — old style, avoid it

- ☐ DATA TYPES — practice each in script.js:
 - String: **let greeting = "Hello World";**
 - Number: **let price = 15000;**
 - Boolean: **let isOnline = true;**
 - Array: **let skills = ["HTML", "CSS", "JavaScript"];**
 - Object: **let intern = { name: "Ali", week: 2, city: "Lahore" };**

- ☐ FUNCTIONS — write each type in script.js:
 - Regular: **function greet(name) { return "Hello " + name; }**
 - Arrow: **const greet = (name) => "Hello " + name;**

- ☐ LOOPS — practice these patterns:
 - **for (let i = 0; i < 5; i++) { console.log(i); }**
 - **skills.forEach(skill => console.log(skill));** — loop through array

- ☐ CONDITIONALS — write one if/else:

```
→ if (isOnline) { console.log("User is online"); } else { console.log("Offline"); }
```

☐ TEMPLATE LITERALS — the modern way to build strings:

```
→ console.log(`Welcome to ${site}, ${name}!`);
```

☐ Commit: `git add . && git commit -m "Day 10: JavaScript fundamentals"`

☐ Post console screenshots showing your code running: "Day 10 Done ■"

LEARNING RESOURCES — DAY 3

YT

JavaScript Crash Course

youtube.com/watch?v=hdI2bq0jy3c

Traversy Media — JS for beginners, 90 minutes

YT

JS in 100 Seconds

youtube.com/watch?v=DHjqpvDnNGE

Fireship — fast JS overview, watch first

W3

W3Schools JavaScript

w3schools.com/js

Reference for every JS concept with live examples

MD

MDN JavaScript Guide

developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

Official in-depth JS documentation

MENTOR TIP — JAVASCRIPT MINDSET

Do not memorize syntax. Understand the **concept**, then look up the syntax when you need it.
Every senior developer googles basic JS syntax daily. The skill is knowing **what** to search for.
The browser console is your best friend. Test every new concept there before adding it to your file.

WEEK 2 — WHAT YOU LEARNED TODAY

→ Understood `let`, `const`, and the 5 core data types (string, number, boolean, array, object)
→ Wrote regular and arrow functions — know the difference
→ Practiced `for` loops and `forEach` on arrays
→ Used template literals for clean string formatting

DAY 1

CSS Grid Layout

DAY 2

Animations & Effects

DAY 3

JS Fundamentals

DAY 4

DOM Manipulation

DAY 5

Fetch API & Review

The DOM (Document Object Model) is JavaScript's way of seeing and changing your HTML. Today you use JS to select elements, change content, listen for clicks, and build a live feature.

YOUR TO-DO LIST — DAY 4

☐ Open your project. Make sure script.js is linked. Open browser Console with F12☐ SELECTING ELEMENTS — practice in the console first:

→ `document.querySelector("h1")` — selects the first h1 on the page

→ `document.querySelectorAll(".card")` — selects ALL elements with class "card"

→ `document.getElementById("hero")` — selects element by ID

☐ CHANGING CONTENT — practice in the console:

→ `document.querySelector("h1").textContent = "Hello from JavaScript!"`;

→ `document.querySelector("h1").style.color = "#00C864"`;

☐ ADDING & REMOVING CLASSES:

→ `document.querySelector(".card").classList.add("highlighted")`;

→ `document.querySelector(".card").classList.remove("highlighted")`;

→ `document.querySelector(".card").classList.toggle("highlighted")`; — toggles on/off

☐ PROJECT — Build a Tab Switcher (Step by Step):

→ Add to HTML: 3 `<button>` tabs and 3 `<div class="tab-content">` panels below them

→ Style tabs in CSS: make active tab have a different color using `.active` class

→ In script.js, select all tab buttons: `const tabs = document.querySelectorAll(".tab-btn")`;

→ Add click event to each: `tabs.forEach(tab => tab.addEventListener("click", () => { ... })))`

→ Inside the click handler: remove `.active` from all tabs, add `.active` to clicked tab

→ Hide all tab content panels, show only the matching one

☐ Test clicking through all 3 tabs — content should switch correctly

☐ Commit: `git add . && git commit -m "Day 11: DOM manipulation and tab switcher"`☐ Post a screen recording of your tab switcher working: "Day 11 Done ■"

LEARNING RESOURCES — DAY 4

YT

DOM Manipulation

youtube.com/watch?v=y17RuWkWdn8

Traversy Media — complete DOM crash course

YT

Event Listeners

youtube.com/watch?v=XF1_MlZ5l6MDani Crossing — `addEventListener` explained clearly

MD

MDN DOM Guide

developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

Full DOM API documentation

JS

JavaScript.info DOM

javascript.info/document

Best written explanation of DOM — free, clear, deep

MENTOR TIP — EVENT LISTENERS

`addEventListener` is the professional way to handle events — never use `onclick=""` inside HTML.

The most common events: **click**, **input**, **submit**, **keydown**, **mouseover**.

Always put JS at the bottom of `<body>` OR use the **defer** attribute — so HTML loads first.

WEEK 2 — WHAT YOU LEARNED TODAY

- Selected DOM elements using `querySelector`, `querySelectorAll`, `getElementById`
- Changed text content, styles, and CSS classes using JavaScript
- Built a working 3-tab switcher — pure HTML, CSS, and JavaScript, no libraries
- Mastered `addEventListener` and event-driven programming pattern

Fetch API & Week 2 Review

DAY 1

CSS Grid Layout

DAY 2

Animations & Effects

DAY 3

JS Fundamentals

DAY 4

DOM Manipulation

DAY 5

Fetch API & Review

Real websites talk to the internet — they fetch data from external APIs (weather, quotes, maps). Today you connect your JS to a real API and do a full review of everything from this week.

YOUR TO-DO LIST — DAY 5

☐ Watch the Fetch API crash course (link below) — 20 minutes

☐ In script.js, add a new section at the bottom for the API mini-app

☐ FETCH BASICS — type this out (don't copy-paste):

```
→ fetch("https://api.quotable.io/random")
```

```
→ .then(response => response.json())
```

```
→ .then(data => console.log(data.content))
```

```
→ .catch(error => console.log("Error:", error));
```

☐ Open browser Console (F12) and refresh — you should see a random quote logged

☐ BUILD — Random Quote Generator App:

```
→ Add to your HTML: a <div id="quote-box">, a <p id="quote-text">, and a <button id="new-quote">New Quote</button>
```

```
→ Style the quote box in CSS — center it, add padding, background, border
```

```
→ In script.js, write a function getQuote() that fetches from the API and updates the p tag
```

```
→ Call getQuote() once when the page loads so a quote shows immediately
```

```
→ Add: document.getElementById("new-quote").addEventListener("click", getQuote);
```

```
→ Add a loading state: show "Loading..." in the p tag while fetch is running
```

☐ WEEK 2 REVIEW CHECKLIST — make sure you can answer these without looking:

```
→ What is the difference between CSS Grid and Flexbox?
```

```
→ How do you add a transition effect? What does "ease" mean?
```

```
→ What is the difference between let and const?
```

```
→ How do you select all elements with a class using querySelector?
```

→ What does `.then()` do in a fetch call?

- ☐ Push all your Week 2 work: **git push origin main**
- ☐ Update your GitHub Pages site — your live site should now have all Week 2 features
- ☐ Post your live GitHub Pages link in the community: "Week 2 Complete! 🟩"

LEARNING RESOURCES — DAY 5

YT

Fetch API Crash Course

youtube.com/watch?v=cuEtnrL9-H0

Traversy Media — Fetch API for beginners

YT

Async/Await Explained

youtube.com/watch?v=V_Kr90SfDeU

Fireship — async/await in 100 seconds

QT

Quotable API

api.quotable.io/random

Free quotes API — no key needed, great for practice

JS

JavaScript.info Fetch

javascript.info/fetch

In-depth Fetch API guide with real examples

MENTOR TIP — APIs & REAL WORLD

Every modern web app uses APIs. Once you understand Fetch + `.then()`, you can connect to **any** service. The pattern is always the same: fetch URL → parse JSON → update the DOM. Master this loop. Next week in React, you will use the same Fetch logic inside `useEffect` — it all connects.

WEEK 2 — WHAT YOU LEARNED TODAY

- Used the Fetch API to pull live data from a real external service
- Built a Random Quote Generator — fetch, display, update on click
- Added a loading state — professional UX pattern used in all real apps
- Reviewed and solidified all 5 days of Week 2 concepts

CSS & JavaScript Cheatsheet

CSS GRID REFERENCE

`display: grid;` — Enables CSS Grid on a container

`grid-template-columns: repeat(3, 1fr);` — Creates 3 equal...

`grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));`

`gap: 24px;` — Space between all grid items

`grid-column: span 2;` — Item stretches across 2 columns

`grid-row: span 2;` — Item stretches across 2 rows

`justify-items: center;` — Aligns grid items horizontally inside c...

`align-items: center;` — Aligns grid items vertically inside cells

JAVASCRIPT ESSENTIALS

`const x = 5;` — Declare constant — cannot be reassigned

`let y = "hello";` — Declare variable — can be reassigned

``Hello ${name}!`` — Template literal — embed variables in strin...

`[1,2,3].forEach(n => console.log(n));` — Loop through ever...

`[1,2,3].map(n => n * 2);` — Create new array by transforming...

`[1,2,3].filter(n => n > 1);` — Create new array keeping onl...

`const { name, age } = person;` — Destructure object — extrac...

`const arr2 = [...arr1, 4, 5];` — Spread operator — copy an...

CSS ANIMATIONS REFERENCE

`transition: all 0.3s ease;` — Smooth transition on any chan...

`transition: color 0.3s, transform 0.3s;` — Transition spec...

`transform: translateY(-6px);` — Moves element 6px upward (...)

`transform: scale(1.05);` — Enlarges element slightly on hover

`animation: name duration timing;` — Applies a @keyframes a...

`animation-iteration-count: infinite;` — Loops the animati...

`@keyframes fadeIn { from {...} to {...} }` — Defines a cu...

`animation-fill-mode: forwards;` — Keeps end state after anim...

DOM API REFERENCE

`document.querySelector(".class")` — Select first matching ele...

`document.querySelectorAll(".class")` — Select all matching e...

`element.textContent = "text";` — Set plain text content of ele...

`element.innerHTML = "<b>bold</b>";` — Set HTML content (us...

`element.classList.add("active");` — Add a CSS class to ele...

`element.classList.toggle("open");` — Toggle class on / off

`el.addEventListener("click", fn);` — Listen for a click event

`element.style.color = "#00C864";` — Set an inline CSS style ...

QUESTIONS? GET STUCK? REACH OUT.

WhatsApp Community · growthliftpk@gmail.com · 0328 0055453

Abdul Rehman (Founder) · Asad Riaz (CTO) · growthlift-digital.vercel.app